Signal Processing Project

MID-TERM PROGRESS REPORT

1. Project Details

- **Project Title:** Distributed Acoustic Sensing for Predictive Machinery Maintenance
- **Software Codename:** MachineWhisperer
- **Author:** Krishna Gorai (Roll No: 25f1100001)
- **Course:** Signal Processing Project (EE4999)
- **Github repo:** [Machine-whisperer]

2. Introduction

MachineWhisperer is a real-time acoustic sensing and predictive maintenance platform being developed as a custom, hardware-integrated signal processing project. The core objective is to monitor machinery noise, process the audio using Digital Signal Processing (DSP) techniques, and visualize the equipment's health to detect potential mechanical failures before they become critical.

Currently, to facilitate rapid development of the DSP pipeline and dashboard, the software utilizes a PC/laptop microphone for real-time data capture. The processing includes filtering out mains hum, generating a continuous spectrogram, tracking RMS energy, and using a baseline RMS threshold for basic anomaly detection.

3. Objectives

- Develop a robust DSP pipeline to process acoustic data.
- Implement real-time audio filtering (Butterworth bandpass) to isolate mechanical frequencies.
- Compute continuous Short-Time Fourier Transforms (STFT) for spectrogram visualization.
- Track Root Mean Square (RMS) energy as a baseline health indicator.
- Build a responsive, industrial dashboard using PyQtGraph.
- Incorporate automated acoustic calibration to suggest optimal filter boundaries.
- *Future:* Integrate an ESP32 edge device for wireless audio capture.
- *Future:* Implement Machine Learning classifiers for advanced anomaly detection.

4. Proposed System Architecture

The final proposed system will process an input acoustic signal captured at the edge through multiple stages:

Hardware Edge Node (ESP32 + I2S Mic) ⟶ TCP/IP Stream ⟶ Backend DSP Server (Python) ⟶

Signal Filtering & STFT ⟶ RMS Calculation & Anomaly Detection ⟶ PyQtGraph Dashboard

Note: The current mid-term implementation uses the local laptop microphone in place of the hardware edge node to allow for DSP and UI development.

5. Technologies Used

Current Implementation (Phase 1 — Mid-Term)

Technology	Purpose
Python 3	Core programming language.
NumPy	High-performance buffer management, Real FFT (rfft), Hanning windowing, and continuous RMS energy calculations.
SciPy (scipy.signal)	4th-order Butterworth bandpass filter design and Welch's Power Spectral Density (PSD) estimation.
sounddevice	Low-latency audio stream ingestion at 16 kHz sample rate (currently local mic).
PyQt6	SCADA-style desktop user interface layout and event architecture.
PyQtGraph	Hardware-accelerated dynamic graphics rendering (Spectrograms, Waveforms, RMS Curves).

Future Implementation (Phase 2)

Technology	Purpose
ESP32 & I2S Mic	Remote edge hardware for raw acoustic capture.
C++ & Sockets	Firmware for data packetization and TCP/IP streaming.
scikit-learn / TensorFlow Lite	Machine learning models for advanced fault classification based on acoustic features.

6. Implementation Progress

6.1 Acoustic Calibration Tool

A standalone scanner (environment_scanner.py) has been developed. It samples room acoustics for 20 seconds using Welch's method to suggest optimal Butterworth bandpass limits based on the Power Spectral Density (PSD). It includes a hard floor of 80Hz to filter out 50Hz electrical mains hum.

Status: Completed

6.2 Live DSP Pipeline

The core processing engine is functional. It performs real-time audio filtering using a Butterworth bandpass filter. It computes the FFT (utilizing a Hanning window to prevent edge leakage) and calculates the RMS power of the filtered signal continuously.

Status: Completed

6.3 Dynamic Visualizations (Dashboard)

A dark-themed, industrial-style GUI has been built using PyQt6 and PyQtGraph. It features:

- A continuous, scrolling 'inferno' spectrogram for frequency activity over time.
- A live filtered acoustic waveform plot with dynamic Y-axis scaling for loud transients.
- An RMS energy trend line plotting machine health.
- Manual frequency input controls (spinboxes) for precision filter tuning.

Status: Completed

6.4 Anomaly Detection

Basic statistical threshold logic has been added. It triggers visual UI alerts (e.g., flashing graphs and warning text) when the real-time RMS energy spikes above a defined baseline.

Status: Completed (Basic Thresholding)

7. Current Results

The system successfully captures live audio, filters out low-frequency hum, and visualizes the clean signal. The dynamic Y-axis scaling works correctly to accommodate transient noises (like claps), and the inferno spectrogram clearly shows frequency changes over time. The basic anomaly detection correctly flags when the RMS threshold is breached.

(Please refer to the attached dashboard video demo GIF)

8. Remaining Work & Future Workflow

The project is currently transitioning from Phase 2 (Core DSP and UI) to Phase 3 (Hardware Integration and Advanced Analytics).

- **Hardware Integration:** Write C++ code for the ESP32 to capture I2S audio from the digital microphone and stream it continuously via a TCP socket to the Python server, replacing the local laptop microphone.
- **Machine Learning Integration:** Replace the current hardcoded RMS threshold with a trained Machine Learning classifier (e.g., SVM or Random Forest) using extracted features (like MFCCs) to categorize specific mechanical faults.
- **UI Updates:** Activate the "Under Development" sections of the dashboard (Machine Health scores and specific anomaly classification) to reflect the dynamic outputs of the ML model.

9. Conclusion

The MachineWhisperer project has made significant progress, successfully implementing a robust software-based DSP pipeline and a high-performance visualization dashboard. By starting with local audio capture, the core requirements of filtering, continuous STFT, and RMS tracking have been achieved. The project is well-positioned for the upcoming hardware integration and machine learning phases.